

Localizing Hot Pixels in Android Malware Images

From image-based detection to class-level malicious code localization

R. Can Oztas · Sevil Sen

WISE Lab, Hacettepe University

EAI SecureComm 2026 · Lancaster University, UK



Motivation

Android malware: a growing, opaque threat

- 33.3M attacks blocked and 1.1M new malicious installers in 2024 alone [Kaspersky '24]
- Detection is largely solved; the analyst question is not:
“*Which class inside this APK is malicious, and why?*”
- Image-based CNNs [Nataraj '11, DexRay '21] treat DEX bytes as pixels but stop at *APK-level* verdicts.
- XAI heatmaps [ImageDroid '23] highlight regions but never resolve them to inspectable code.

The gap

Bridge **image-based detection** → **class-level, analyst-facing code localization**, fully statically.

Our proposal

Hot Pixel Localization (HPL) —
CNN + XAI ensemble + an *invertible*
pixel→byte→Smali map.

Contributions

1. A **fully-static** pipeline that unifies image-based detection with *class-level* XAI-driven localization.
2. Thorough evaluation on *both* large-scale detection and class-level localization, using *real* and *synthetic* malware.
3. An **auditable** support-region-based mapping from heatmap $\rightarrow$ DEX byte-range $\rightarrow$ Smali class $\rightarrow$ analyst-inspectable artifact.
4. Public reference implementation:

`github.com/WiseLabX/HPL`

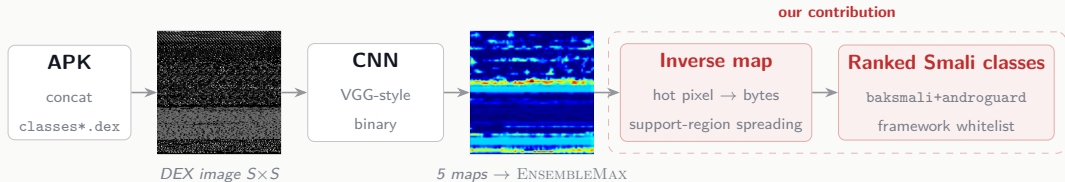
Where HPL sits in the landscape

Work	Representation	Localization	Static	Maps to Smali
DexRay [6]	DEX image	APK (detection)	✓	—
ImageDroid [27]	DEX image + Grad-CAM	visual region	✓	—
LensDroid [29]	multi-view behaviours	component ranking	mixed	partial
Droidetec [31]	API / opcode seq.	API / method	✓	✓
MKLDroid [30]	multi-view graphs	class	✓	✓
XAIDroid [18]	API-call graph (GAT)	class / method	✓	✓
HPL (ours)	DEX image + XAI ensemble	class	✓	✓

- Image-based works stop at region-level explanations.
- Sequence / graph works reach classes but forgo the 2-D structural view.
- HPL is the first to **bridge image XAI to Smali-level artifacts**.

Method

HPL pipeline



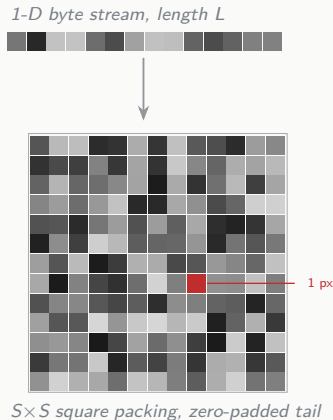
Detection gives the *signal*; the inverse map is what turns that signal into something an analyst can open in a disassembler.

Step 1 — DEX-to-image, with a mappable coordinate system

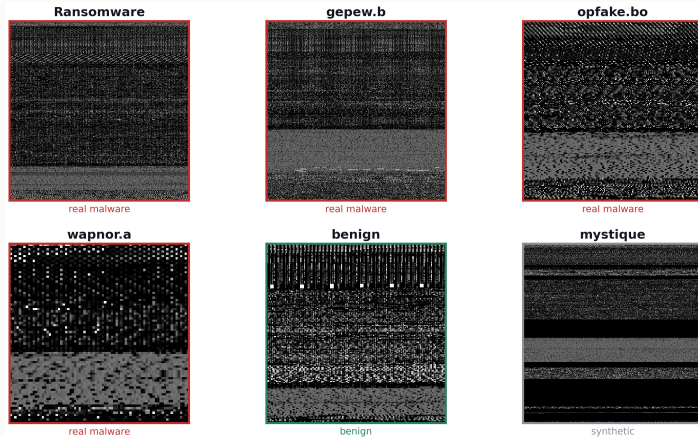
- Concatenate all `classes*.dex`: 1-D byte stream of length L .
- Reshape into a *sample-specific* square $S \times S$ grid, $S = \lceil \sqrt{L} \rceil$; zero-pad.
- Resize to **fixed** $S_0 \times S_0$ ($S_0=256$) for the CNN.
- Persist the triple (S, S_0, L) per sample — this is what lets us invert every heatmap coordinate back to a DEX byte offset.

Design decision

The CNN input is fixed; the intermediate $S \times S$ grid exists only to preserve a *traceable* square packing.



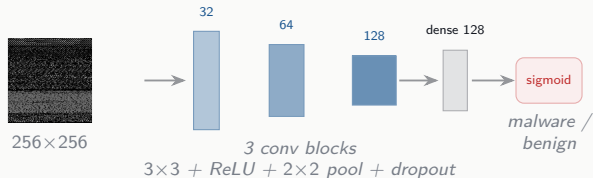
What DEX images actually look like



Structural bands are visible to the eye — and, as the CNN shows, learnable.

Step 2 — CNN classifier

Architecture (VGG-style)



Adam, LR 10^{-3} , BCE loss, 100 epochs.

Training set. KronoDroid [16] — 41,382 malware + 36,755 benign; 5-fold CV.

Localization evaluation datasets

- **Real:** 105 XAIDroid [18] samples — 7 families $\times$ 15.
- **Synthetic:** Mystique [19] — 10 payloads injected into 10 benign carriers.

Why binary + attribution rather than multi-class?

Malicious *regions*, not families, drive analyst triage; attribution transfers across families.

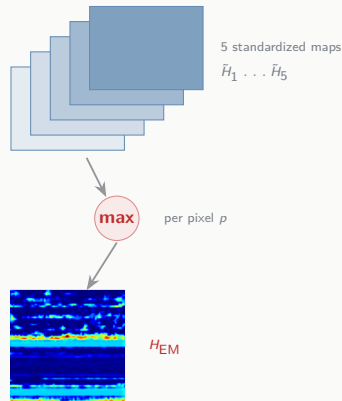
Step 3 — five attribution maps, one Ensemble Max

CAM family (coarse, structural)

- **Grad-CAM++** — weighted last-conv gradients.
- **Score-CAM** — gradient-free, confidence-weighted.

Gradient family (fine, pixel-level)

- **Vanilla Saliency** — $\partial y / \partial x$.
- **SmoothGrad** — noise-averaged saliency.
- **Integrated Gradients** — path integral from baseline.



$$H_{EM}(p) = \max_{k \in \{1, \dots, 5\}} \tilde{H}_k(p)$$

Robust to any single method's failure mode.

Feeds a **per-class score list**, not just a map.

Step 4 — Pixel $\rightarrow$ byte range $\rightarrow$ Smali class

Inverse coordinate transform. For a hot-pixel centroid (x_0, y_0) on the resized $S_0 \times S_0$ heatmap:

$$x = \left\lfloor x_0 \cdot \frac{S}{S_0} \right\rfloor, \quad y = \left\lfloor y_0 \cdot \frac{S}{S_0} \right\rfloor, \quad \text{idx} = y \cdot S + x$$

Support-region max-spreading.

Resize is lossy $\rightarrow$ each hot pixel covers a byte-*interval* $[b_{\min}, b_{\max}]$. We take the max saliency across the interval instead of interpolating.

Round-trip sanity check [105 XAIDroid samples]

- Class-match hit rate: **1.00**
- Positional error (median / p95): **0 / 0 bytes**
- Support-region width (median / p95): **261 bytes**

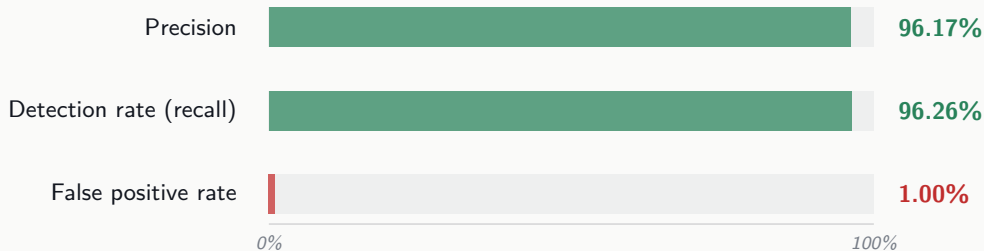
From bytes to code

Disassemble with `baksmali`, parse DEX with `androguard`, resolve the byte range to a Smali class, then filter against a benign-library whitelist (AndroidX, Google services, okhttp3, ...).

Results

Detection performance — KronoDroid

HPL — VGG-style CNN on KronoDroid, 5-fold CV



- Confirms the DEX-image representation is competitive at scale — a solid *foundation* for XAI-based localization.
- Detection is not the contribution; it is the **substrate** the attribution methods interpret.

Localization performance — AUC & NSS

Method	Real (XAIDroid)		Synthetic (Mystique)	
	AUC	NSS	AUC	NSS
Score-CAM	0.86	1.50	0.86	1.45
Grad-CAM++	0.87	1.62	0.88	1.65
Vanilla Saliency	0.89	1.72	0.91	1.80
SmoothGrad	0.87	1.64	0.90	1.54
Integrated Gradients	0.55	0.55	0.61	0.73
Ensemble Max	0.89	1.71	0.89	1.49

- **Ensemble Max** is the most consistent method across both real and synthetic malware.
- **Integrated Gradients** struggles: path integration is designed for smooth natural images, not discrete bytecode.
- Localization quality tracks payload *contiguity*: contiguous families (*apofers.b*, *opfakes.bo*) localize cleanest.

Case studies

From a hot pixel to a class the analyst can open



The evaluation question (AUC / NSS)

- Does the heatmap land on ground-truth malicious classes?

The analyst question (next three slides)

- Is the top-ranked class *actually useful* when you open it?
- We manually reverse-engineered the top hits for three families.

Case 1 — *agent.aax*: HPL beats its own ground truth

HPL ranked output

#1

Lcom/software/app/R

not in the ground-truth set

Family. *Trojan-SMS.AndroidOS.Agent.aax*

— sends SMS to premium-rate numbers without consent.

Why it was missed

Ground truth lists *executable* malicious classes. R is a resource-reference class — no executable code, so nobody labelled it.

Non-executable $\neq$ uninteresting. HPL reads **bytes**, so it sees configuration the code-centric ground truth cannot.

What manual analysis found inside it

- `error_sms_sending` — the fraud path's error strings.
- `megafon_rules` — per-operator rules for a *Russian telecom*.
- `register_path` — the **C2 registration** endpoint.
- A pointer into a raw resource holding the **premium-rate numbers and SMS templates** used for the fraud.

In other words: the campaign's entire *configuration*, in one class the label set never contained.

Case 2 — *opfake.bo*: names are gone, structure is not

HPL ranked output

#1

(randomised name)

= the config / constants class

Family. *Trojan-SMS.AndroidOS.Opfake.bo*
— SMS fraud plus fake login overlays on legitimate apps.

The evasion

Every package and class name is replaced with a meaningless random string — defeating signature matching and human skim-reading alike.

What manual analysis found inside it

- **Timing constants** for the malware's scheduled tasks — the orchestration schedule.
- Static strings that are **empty until a decryption routine fills them** at start-up — classic anti-analysis.

HPL not only recovered the core malicious packages, it ranked *first* the one class that explains how the rest is coordinated — a non-obvious but structurally central component.

Renaming defeats signatures, not **byte-level structure**. HPL never reads an identifier.

Case 3 — *wapnor.a*: not just *what*, but *how it starts*

HPL ranked output

#1

SROBWZb

exact ground-truth match — the payload

#2

aqrPxuqGxP

the trigger — see right

Family.

Trojan-Dropper.AndroidOS.Wapnor.a — covertly downloads and installs further payloads.

Rank #1 is the payload the ground truth already knew about. Rank #2 is where it gets interesting.

The trigger class, aqrPxuqGxP

baksmali output, boilerplate <init> elided

```
.class public Lcie/yxldxd/nswll/aqrPxuqGxP;
.super Landroid/app/Application;

.method protected attachBaseContext(Landroid/content/Context;)V
    invoke-super {p0, p1}, Landroid/app/Application;->
    attachBaseContext(Landroid/content/Context;)V

    const-string v0, "KVzWSzWai"
    const-string v1, "CZuMENnIaK"
    invoke-static {p1, v0, v1}, Lcie/yxldxd/nswll/SROBWZb;->b(
        Landroid/content/Context;Ljava/lang/String;Ljava/lang/String;)V
    return-void
.end method
```

Three instructions: call super, then hand two obfuscated strings to **SROBWZb** — rank #1. The dropper fires *before* onCreate, i.e. before any Activity exists.

Ranking is a **narrative**: payload at #1, its ignition mechanism at #2.

Robustness & outlook

Robustness under obfuscation (Obfuscapk)

Three complementary techniques:

- *ArithmeticBranch* — junk / opaque control flow.
- *AdvancedReflection* — reflection wrappers around sensitive APIs.
- *ConstStringEncryption* — encrypts C2 URLs, keys.

Ensemble Max on obfuscated malware:

Metric	mean $\pm$ std
AUC	0.856 $\pm$ 0.068
NSS	1.547 $\pm$ 0.598

- Effects are **family-dependent**: some transformations *disrupt* learned structural patterns; others *introduce* new salient ones (reflection wrappers, decryption stubs).
- Localization remains reliable overall — the ensemble absorbs individual explainer failures.

Limitations and scope

- **Ground truth.** No public pixel-level malicious masks in DEX images; we derive them from class-level annotations.
- **DEX-only.** Native code, packing, and heavy dynamic loading reduce the visibility of malicious logic in the DEX stream.
- **Obfuscation coverage.** Three Obfuscapk transformations; may not generalize to all families.
- **Aggregation trade-offs.** Ensemble Max inherits any explainer's isolated false positives → future work: agreement-based ensembling.
- **Concept drift.** Like all learned malware detectors, requires periodic threshold recalibration and retraining [Transcend '17].

Take-away

- **HPL** is a fully-static pipeline that turns image-based CNN detection into *class-level*, analyst-inspectable code localization.
- Key ingredient: an *invertible* DEX-to-image transform + support-region back-projection — **every heatmap coordinate is auditable**.
- **96%** APK-level detection (KronoDroid); **AUC 0.89 / NSS 1.71** class-level localization on real malware.
- The case studies are the real argument: HPL surfaced a *configuration class the ground truth omitted*, held up under *full identifier randomisation*, and recovered a dropper's *ignition mechanism* — not just its payload.

github.com/WiseLabX/HPL

Thank you.

Questions?